
Algorithmes exacts et approchés pour l'optimisation multi-objectifs

Partie 2

olivier.spanjaard@lip6.fr
M2 AI2D - MADMC - 2025

Plan

1. Problèmes bi-objectifs
 - Énumération ordonnée
 - Méthodes en deux phases
 2. Branch and bound multi-objectifs
 - Borne inférieure multi-objectifs
 - Borne supérieure multi-objectifs
 3. Algorithmes approchés
 - ϵ -dominance
 - grille logarithmique
-

Enumération ordonnée (cas bi-objectifs)

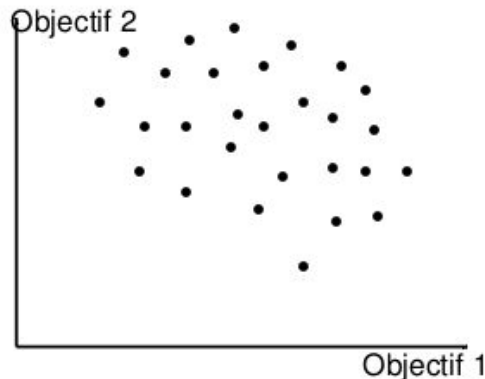
Le cas bi-objectifs

Définition : problème combinatoire bi-objectifs

- E : ens fini d'éléments,
- $c_1(.)$ et $c_2(.)$: deux fonctions objectifs $E \rightarrow \mathbb{Z}^+$,
- $\mathcal{F} \subseteq 2^E$: ens de solutions réalisables,
- $\forall S \in \mathcal{F}, c_i(S) = \sum_{e \in S} c_i(e)$.

But : ens $\mathcal{F}^*$ des solutions Pareto-optimales

Représentation graphique commode de l'espace des objectifs :

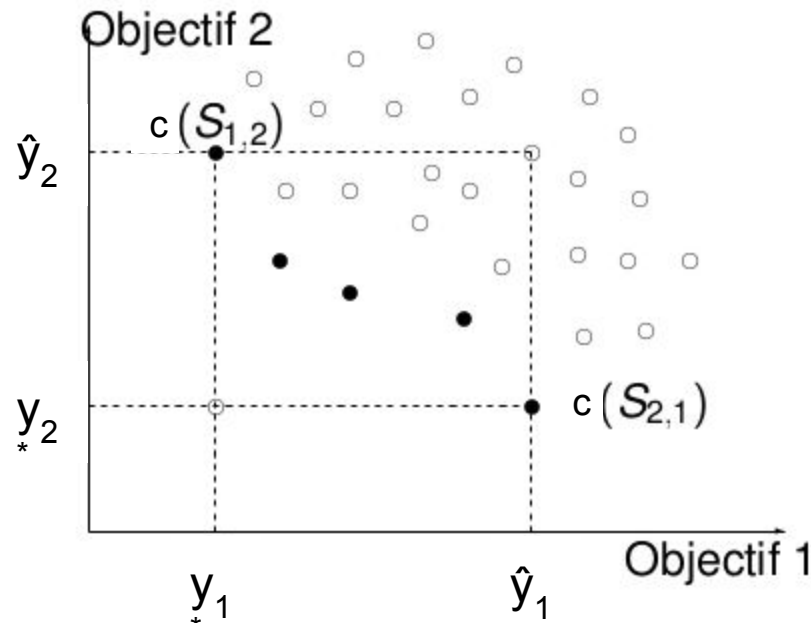


Point idéal et point nadir

Définition : point idéal et point nadir

Le **point idéal** est le vecteur y^* défini par : $y_i^* = \min_{S \in \mathcal{F}} c_i(S)$

Le **point nadir** est le vecteur $\hat{y}$ défini par : $\hat{y}_i = \max_{S \in \mathcal{F}^*} c_i(S)$



Algorithme Climaco et Martins (82)

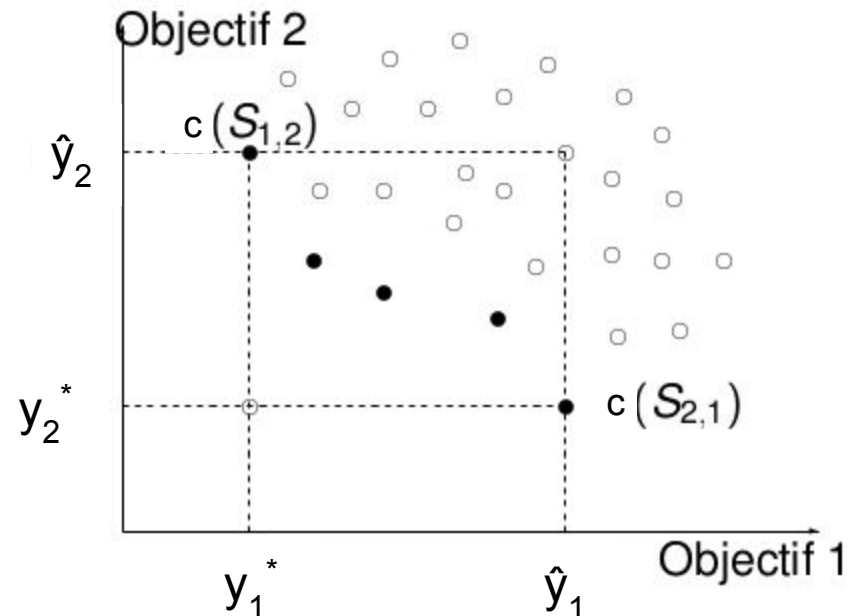
Proposition

Une solution non-dominée S vérifie :

$$y_1^* \leq c_1(S) \leq \hat{y}_1 \text{ et } y_2^* \leq c_2(S) \leq \hat{y}_2$$

Idée de la preuve : Une solution S telle que $c_1(S) > \hat{y}_1$ est dominée par $S_{2,1}$.

Principe de l'algorithme : Commencer avec la solution leximin $S_{1,2}$ puis construire la seconde selon $c_1, c_2 \dots$ jusqu'à atteindre $S_{2,1}$.



Un exemple : MOSS

INSTANCE

objet	objectif 1	objectif 2
1	1	3
2	2	2
3	2	3
4	3	1
5	3	2
6	4	4

Choisir 3 objets parmi 6
20 solutions réalisables

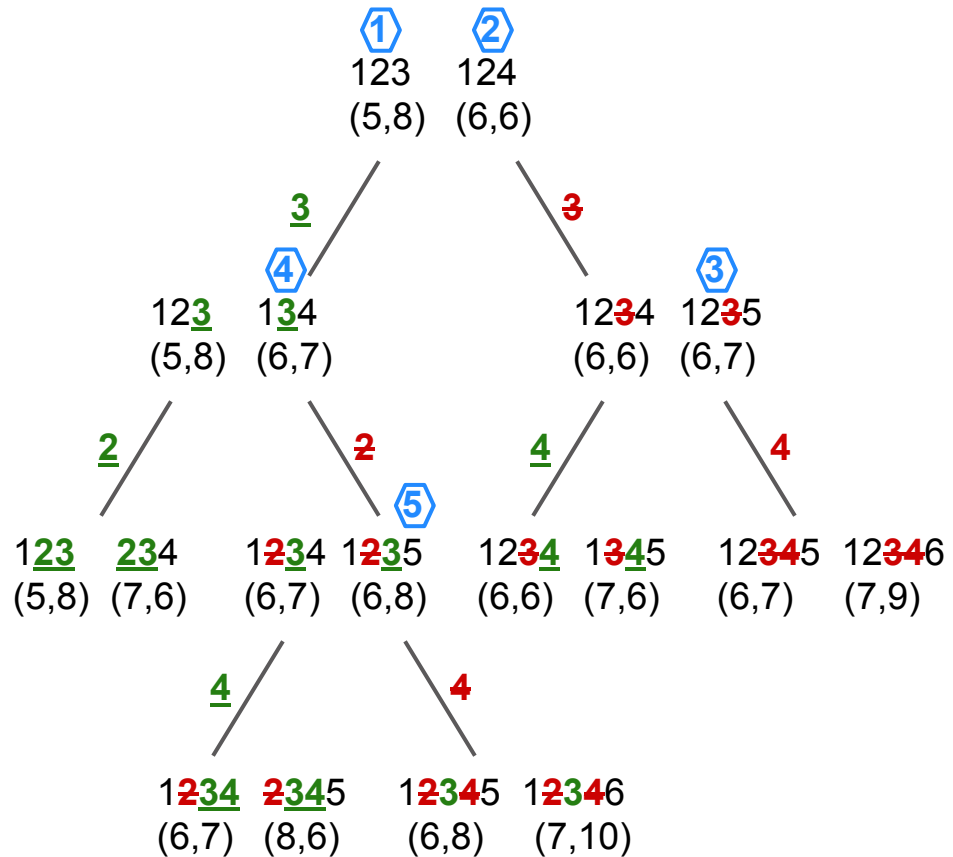
$$\hat{y} = (8, 8)$$

Énumération ordonnée

123	→	(5,8)	1
124	→	(6,6)	2
125	→	(6,7)	3
134	→	(6,7)	4
135	→	(6,8)	5
145	→	(7,6)	
234	→	(7,6)	
235	→	(7,7)	
126	→	(7,9)	
136	→	(7,10)	
245	→	(8,5)	

$345 \rightarrow (8,6)$
 $146 \rightarrow (8,8)$
 $156 \rightarrow (8,9)$
 $236 \rightarrow (8,9)$
 $246 \rightarrow (9,7)$
 $256 \rightarrow (9,8)$
 $346 \rightarrow (9,8)$
 $356 \rightarrow (9,9)$
 $456 \rightarrow (10,7)$

Arbre d'énumération ordonnée



Enumération ordonnée des K meilleures solutions (K meilleurs)

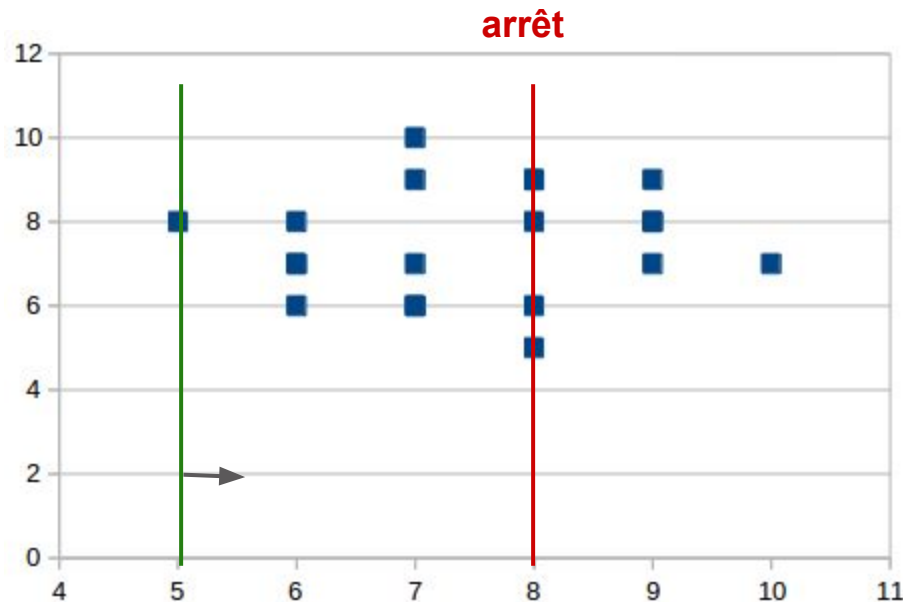
Algorithme inspiré de Gabow (77), Hamacher & Queyranne (85)

1. $S \leftarrow$ solution de coût minimal
2. $Sol \leftarrow \{S\}$
3. $IN \leftarrow \emptyset$; $OUT \leftarrow \emptyset$
4. empiler(S, IN, OUT)
5. $k \leftarrow 1$
6. **tant que** $k < K$ **faire**
 - 6.1. $(S, IN, OUT) \leftarrow$ dépiler()
 - 6.2. $(i, j) \leftarrow$ échange(S, IN, OUT) // $i \in S, j \notin S$
 - 6.3. $S_k \leftarrow S \cup \{j\} \setminus \{i\}$
 - 6.4. $Sol \leftarrow Sol \cup \{S_k\}$
 - 6.5. empiler($S, IN \cup \{i\}, OUT$)
 - 6.6. empiler($S_k, IN, OUT \cup \{j\}$)
 - 6.7. $k \leftarrow k + 1$

- **IN**: élément obligatoires
- **OUT**: éléments interdits
- échange(S, IN, OUT):
détermine le meilleur échange à réaliser

Sortie : Sol , l'ensemble des K meilleures solutions

Limite de l'approche de Climaco et Martins



Inconvénient de l'approche de Climaco et Martins : la condition d'arrêt se déclenche très tardivement.

Une approche plus efficace

Idée : parcourir les solutions dans la direction orthogonale au segment liant les deux points leximin (5,8) et (8,5).

INSTANCE (RETRIEE)

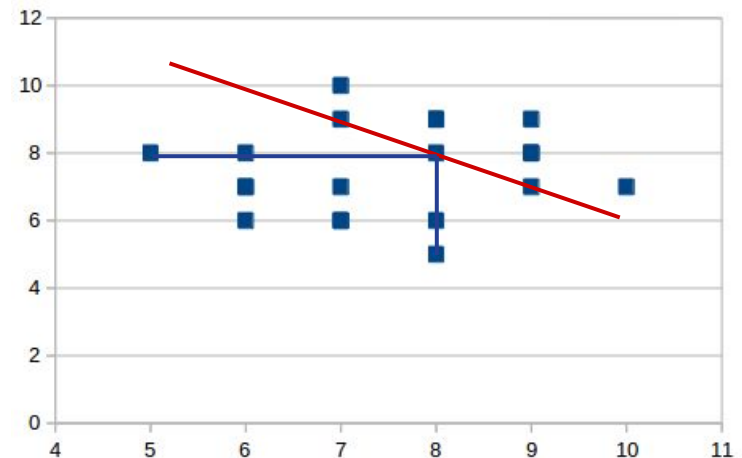
objet	objectif 1	objectif 2
1	1	3
2	2	2
4	3	1
3	2	3
5	3	2
6	4	4

Choisir 3 objets parmi 6
20 solutions réalisables

Enumération ordonnée

124 → (6,6), 12
123 → (5,8), 13
125 → (6,7), 13
143 → (6,7), 13
145 → (7,6), 13
243 → (7,6), 13
245 → (8,5), 13
135 → (6,8), 14
235 → (7,7), 14
435 → (8,6), 14
126 → (7,9), 16
146 → (8,8), 16
246 → (9,7), 16

136 → (7,10), 17
156 → (8,9), 17
etc.



Une approche plus efficace

Idée : parcourir les solutions dans la direction orthogonale au segment liant les deux points leximin (5,8) et (8,5).

INSTANCE (RETRIEE)

objet	objectif 1	objectif 2
1	1	3
2	2	2
4	3	1
3	2	3
5	3	2
6	4	4

Choisir 3 objets parmi 6
20 solutions réalisables

Enumération ordonnée

124 → (6,6), 12

123 → (5,8), 13

125 → (6,7), 13

134 → (6,7), 13

145 → (7,6), 13

234 → (7,6), 13

245 → (8,5), 13

135 → (6,8), 14

235 → (7,7), 14

345 → (8,6), 14

126 → (7,9), 16

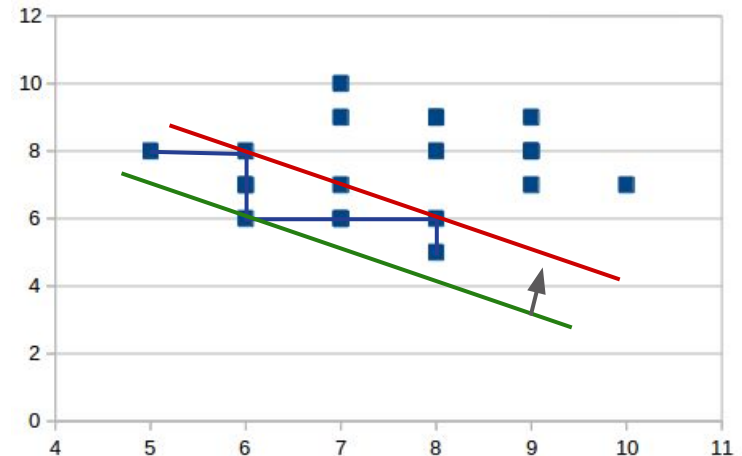
146 → (8,8), 16

246 → (9,7), 16

136 → (7,10), 17

156 → (8,9), 17

etc.



Une approche plus efficace

Idée : parcourir les solutions dans la direction orthogonale au segment liant les deux points leximin (5,8) et (8,5).

INSTANCE (RETRIEE)

objet	objectif 1	objectif 2
1	1	3
2	2	2
4	3	1
3	2	3
5	3	2
6	4	4

Choisir 3 objets parmi 6
20 solutions réalisables

Enumération ordonnée

124 → (6,6), 12

123 → (5,8), 13

125 → (6,7), 13

134 → (6,7), 13

145 → (7,6), 13

234 → (7,6), 13

245 → (8,5), 13

135 → (6,8), 14

235 → (7,7), 14

345 → (8,6), 14

126 → (7,9), 16

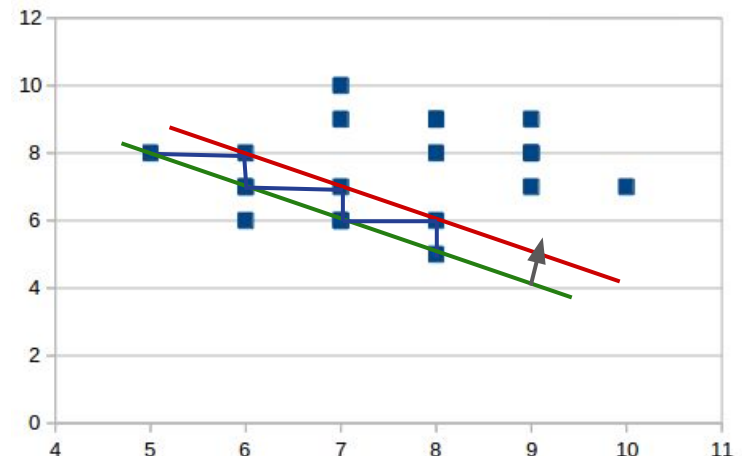
146 → (8,8), 16

246 → (9,7), 16

136 → (7,10), 17

156 → (8,9), 17

etc.



Une approche plus efficace

Idée : parcourir les solutions dans la direction orthogonale au segment liant les deux points leximin (5,8) et (8,5).

INSTANCE (RETRIEE)

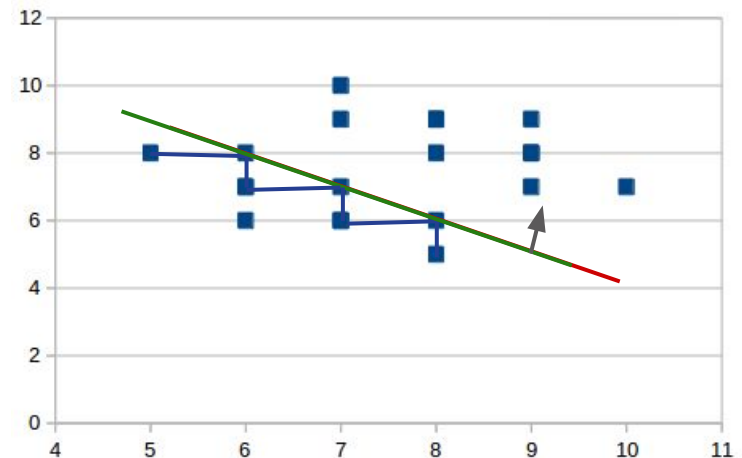
objet	objectif 1	objectif 2
1	1	3
2	2	2
4	3	1
3	2	3
5	3	2
6	4	4

Choisir 3 objets parmi 6
20 solutions réalisables

Enumération ordonnée

124 → (6,6), 12
123 → (5,8), 13
125 → (6,7), 13
134 → (6,7), 13
145 → (7,6), 13
234 → (7,6), 13
245 → (8,5), 13
135 → (6,8), 14
235 → (7,7), 14
345 → (8,6), 14

126 → (7,9), 16
146 → (8,8), 16
246 → (9,7), 16
136 → (7,10), 17
156 → (8,9), 17
etc.



Applications

Ces méthodes peuvent s'appliquer à tout problème d'optimisation combinatoire **pour lequel il existe un algorithme efficace d'énumération ordonnée** des solutions :

- **Plus court chemin** [Yen 1972, Eppstein 1998, Jimenez & Marzal 2003]
- **Arbre couvrant minimal** [Gabow 1977, Katoh et al. 1981, Eppstein 1992]
- **Affectation** [Katoh et al. 1981, Pedersen et al. 2008]
- **Couplage** [Hamacher & Queyranne 1985, Chegireddy & Hamacher 1987]

L'intérêt est bien sûr qu'une fois qu'on dispose de l'algorithme d'énumération ordonnée des solutions, l'adapter pour trouver les solutions Pareto-optimales d'un problème bi-objectifs ne pose pas de difficulté.

Méthodes en deux phases (cas bi-objectifs)

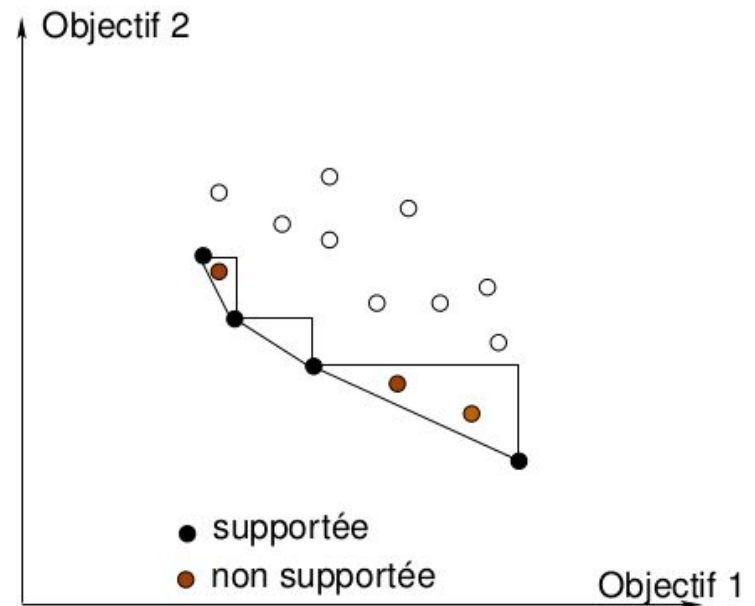
Méthode en deux phases (Ulungu et Teghem, 1993)

- Certaines solutions Pareto-optimales sont plus faciles à calculer que d'autres : ce sont les solutions **supportées**, qui optimisent une combinaison linéaire des objectifs.
- Les autres solutions Pareto-optimales sont dites **non-supportées**.
- Les solutions supportées situées aux sommets de l'enveloppe convexe sont dites **extrêmes**, les autres sont dites **non-extrêmes**.

Méthode en deux phases :

- **première phase** : trouver toutes les solutions **extrêmes**.
- **deuxième phase** : trouver toutes les solutions **non-extrêmes**.

⇒ assez efficace lorsque le problème mono-objectif sous-jacent est “facile” à résoudre.



Optimiser une combinaison linéaire

Optimiser une **combinaison linéaire des objectifs** peut se faire en se ramenant à un problème à un seul objectif car :

$$\begin{aligned}\sum_{i=1}^q w_i c_i(S) &= \sum_{i=1}^q w_i \sum_{e \in S} c_i(e) \\ &= \sum_{i=1}^q \sum_{e \in S} w_i c_i(e) \\ &= \sum_{e \in S} \sum_{i=1}^q w_i c_i(e)\end{aligned}$$

Optimiser une combinaison linéaire des objectifs selon un jeu de poids $w_1, \dots, w_q$ permet d'obtenir une **solution supportée**.

INSTANCE

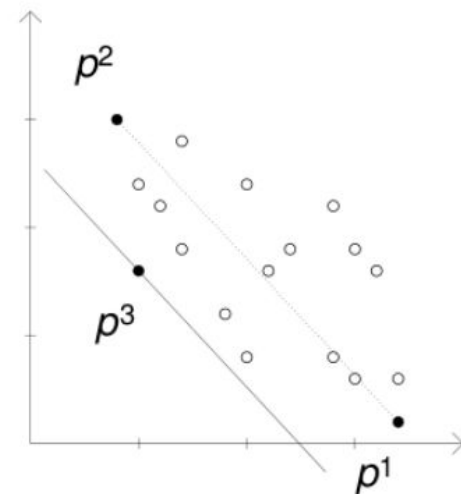
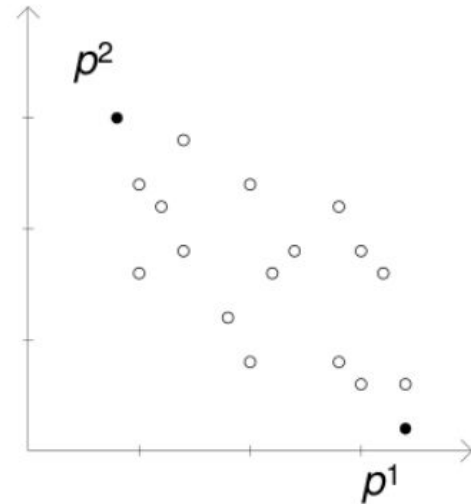
objet	objectif 1	objectif 2	moyenne
1	1	3	2
2	2	2	2
3	2	3	2.5
4	3	1	2
5	3	2	2.5
6	4	4	4

La solution qui minimise $(\text{obj}_1 + \text{obj}_2)/2$ est **{1,2,4}**.

Méthode en deux phases : phase 1

Méthode d'Aneja et Nair :

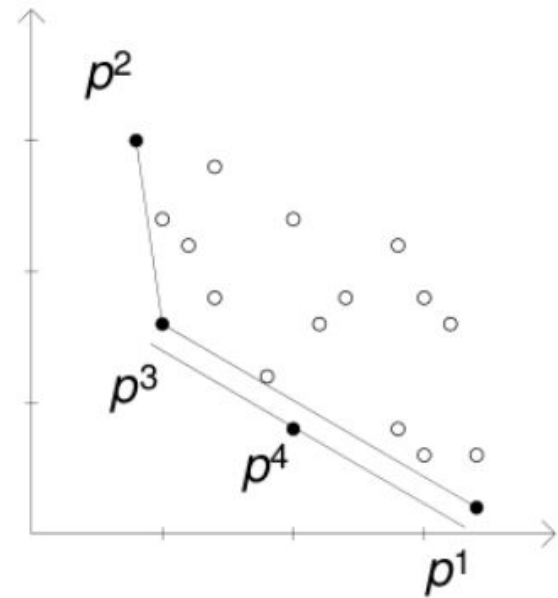
- on identifie les deux points **leximin**
- le calcul de telles solutions revient à de l'optimisation à un seul objectif
- les deux points leximin sont p^1 et p^2
- on calcule **récurivement** les points extrêmes
- combinaison linéaire des objectifs selon la droite qui passe par les points p^1 et p^2
- on trouve le point p^3



Méthode en deux phases : phase 1

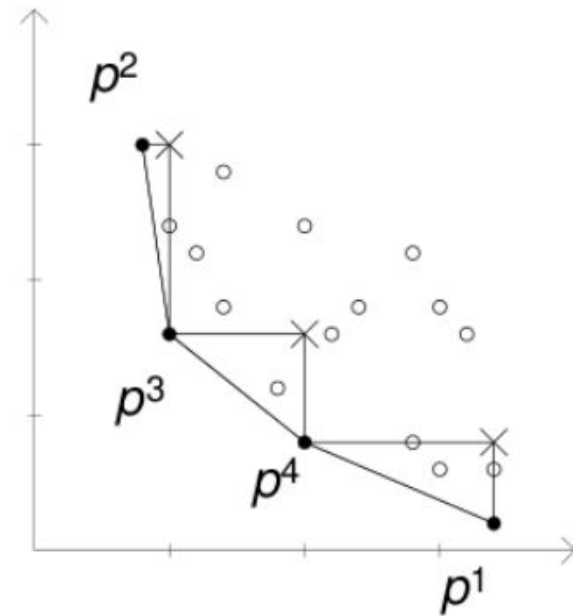
Méthode d'Aneja et Nair (suite) :

- on teste si il existe un point supporté entre p^1 et p^3 , et entre p^3 et p^2
- on trouve le point p^4



Méthode en deux phases : phase 1

A l'issue de la phase 1, les points non-dominés restants se situent nécessairement **dans les triangles** entre deux solutions consécutives.



Pseudo-code de la phase 1

Algorithme d'Aneja et Nair

pour $i=1,2$ **faire**

 ranger lexicographiquement / objectif i

 résoudre le problème pour cet ordre - sol : $s_i = (x_i, y_i)$

si $s_1 = s_2$ **alors retourner** $\{s_1\}$

sinon

$L := \text{RechFront}(s_1, s_2)$

retourner concaténation de $\{s_1\}$, L , $\{s_2\}$

Procédure $\text{RechFront}(s', s'')$

 calculer les nouveaux coûts $\varphi_1(e)(y' - y'') + \varphi_2(e)(x'' - x')$

 résoudre le problème pour ces coûts - sol : $s = (x, y)$

si $s = s'$ ou $s = s''$ **alors retourner** la liste vide

sinon

$L' := \text{RechFront}(s', s)$; $L'' := \text{RechFront}(s, s'')$

retourner concaténation de L' , $\{s\}$, L''

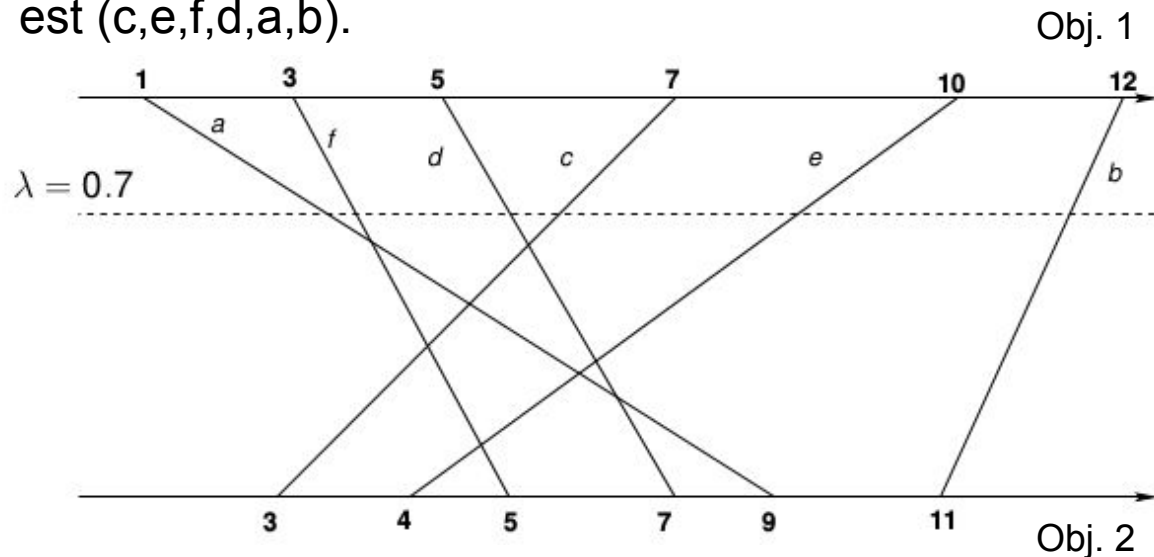
Complexité de la phase 1 pour le problème MOSS

A chaque objet est associé un segment reliant la valeur sur Obj. 1 et la valeur sur Obj. 2.

Le rangement pour $\lambda = 0.7$ est (a,f,d,c,e,b).

Le rangement pour $\lambda = 0.1$ est (c,e,f,d,a,b).

	Obj. 1	Obj. 2
Item a	1	9
Item b	12	11
Item c	7	3
Item d	5	7
Item e	10	4
Item f	3	5

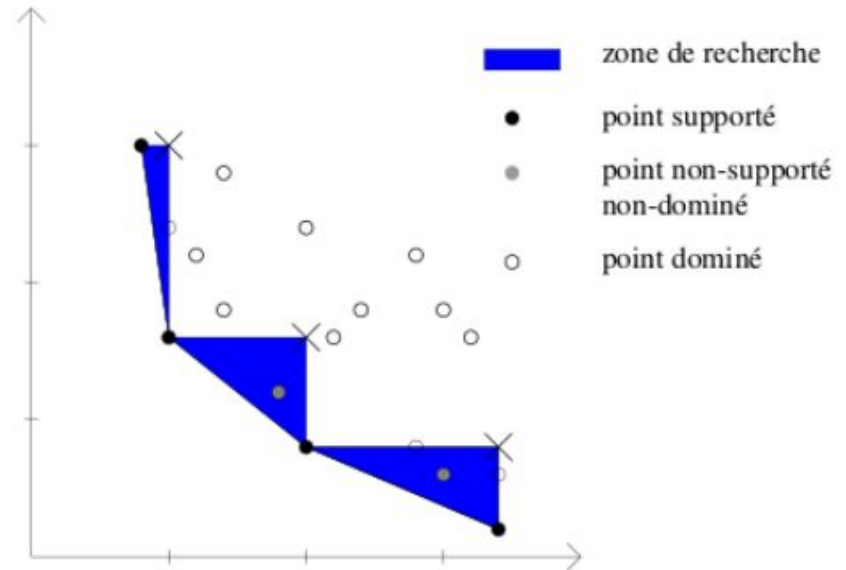


Géométrie algorithmique. Le nombre de points d'intersection est **polynomial** du nombre de segments : $O(n^2)$ où n est le nombre d'objets.

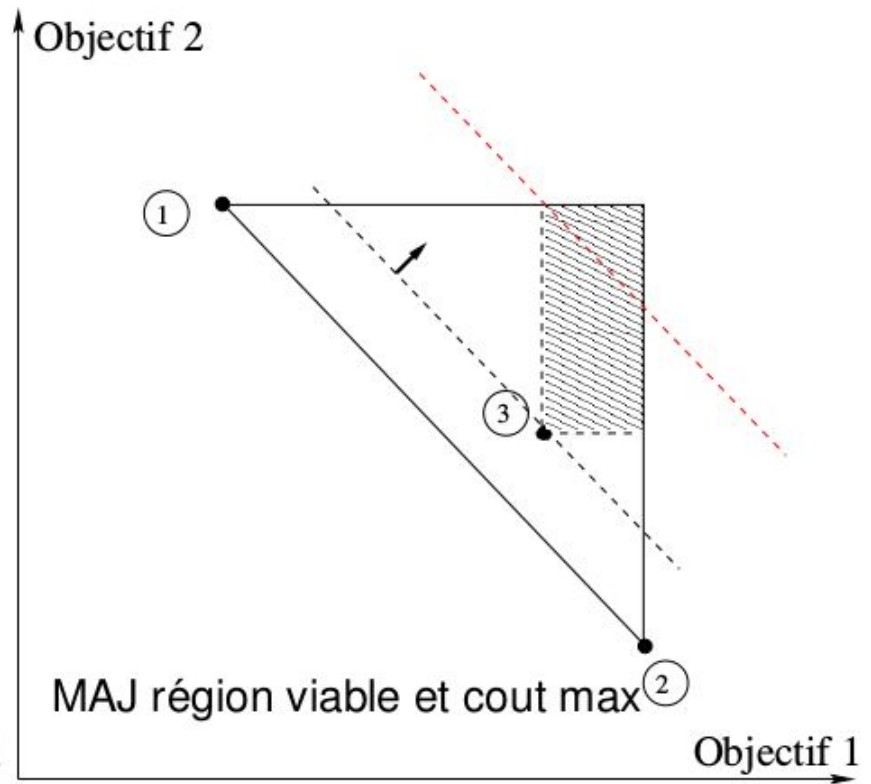
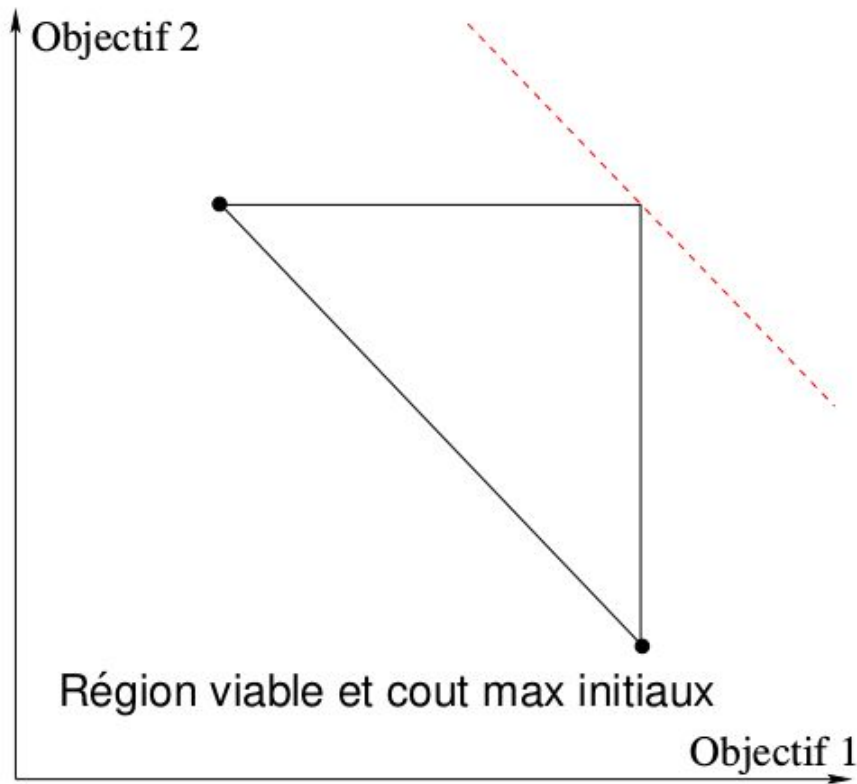
⇒ **nombre polynomial d'appel récursif dans la méthode d'Aneja et Nair.**

Méthode en deux phases : phase 2

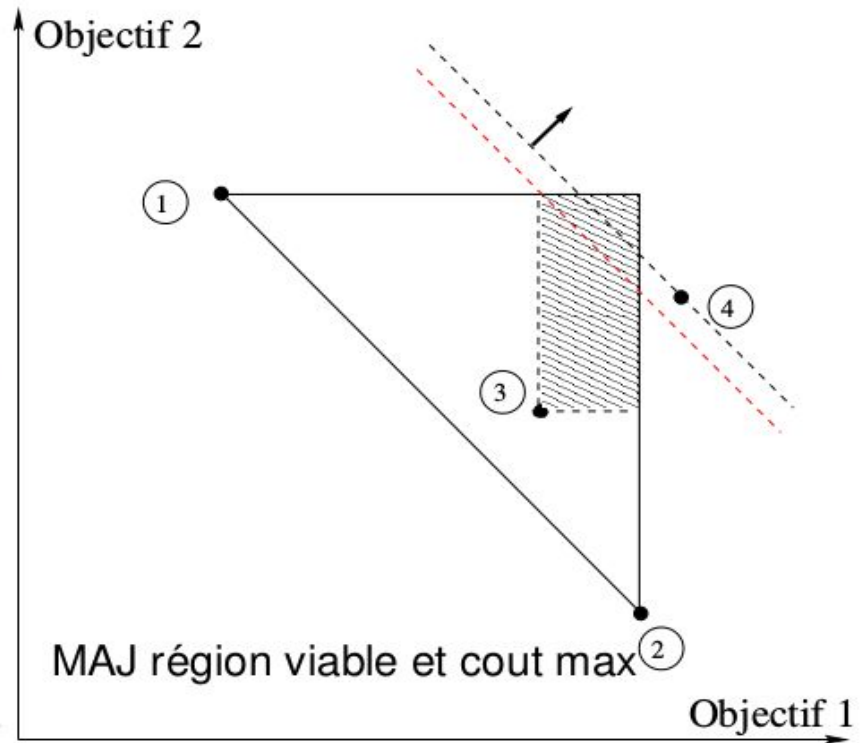
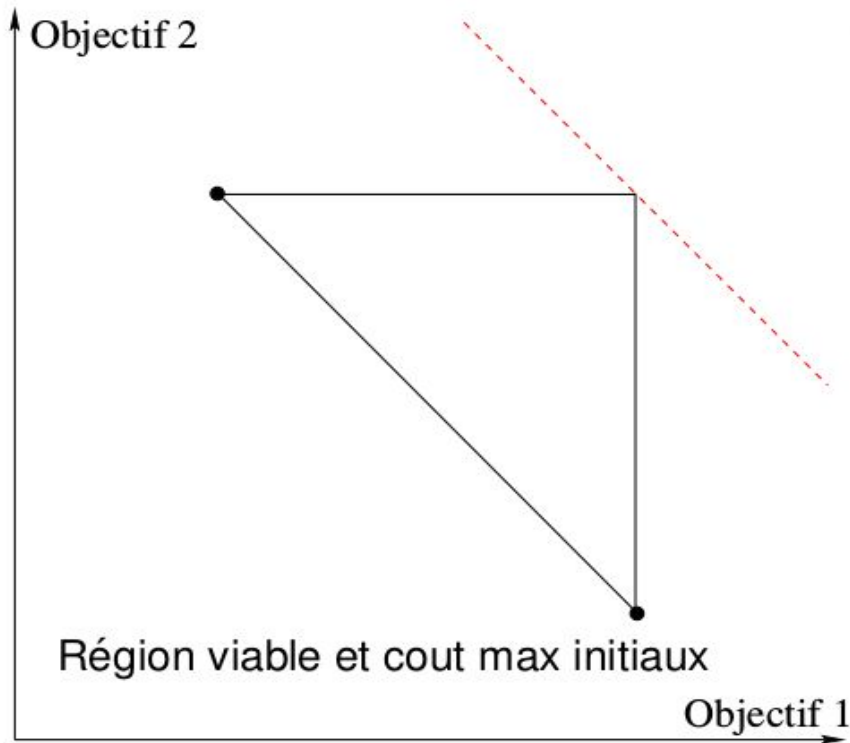
- Explorer les triangles un à un ;
- Version exacte : branch and bound, énumération ordonnée...
- Version approchée : recherche locale



Phase 2 - énumération ordonnée



Phase 2 - énumération ordonnée



Pseudo-code phase 2 - K meilleurs

Algorithme Phase 2

pour tout couple de solutions extrêmes consécutives $s' = (x', y')$,
 $s'' = (x'', y'')$ **faire**
 définir la région viable et le coût maximum
 pour $k = 1, \dots, \infty$ **faire**
 déterminer la k^e meilleure solution $s_k = (x_k, y_k)$
 si il n'y a plus de solution **alors** sortir
 sinon
 si s_k est dans la région viable **alors**
 ajouter s_k à liste des sol. efficaces non-extrêmes
 mettre à jour la région viable et le coût maximum
 sinon si s_k atteint ou dépasse le coût max **alors** sortir

Applications

La méthode en deux phases a été appliquée à la version bi-objectifs de divers problèmes d'optimisation classiques, parmi lesquels :

- **Affectation** [Przybylski et al. 2008, Delort et Spanjaard 2011]
- **Sac à dos** [Visée et al. 1998, Delort et Spanjaard 2010]
- **Flot à coût minimum** [Raith et Ehrgott, 2009]
- **Arbre couvrant minimum**
 - Trois variantes selon la procédure d'exploration des triangles :*
 - une variante approchée [Hamacher et Ruhe 1994] → recherche locale
 - deux variantes exactes [Ramos et al. 1998, Steiner et Radzik 2006] → branch and bound [1998], énumération ordonnée [2006]

Résolution exacte sur des cliques comportant jusqu'à 38 sommets [2006].

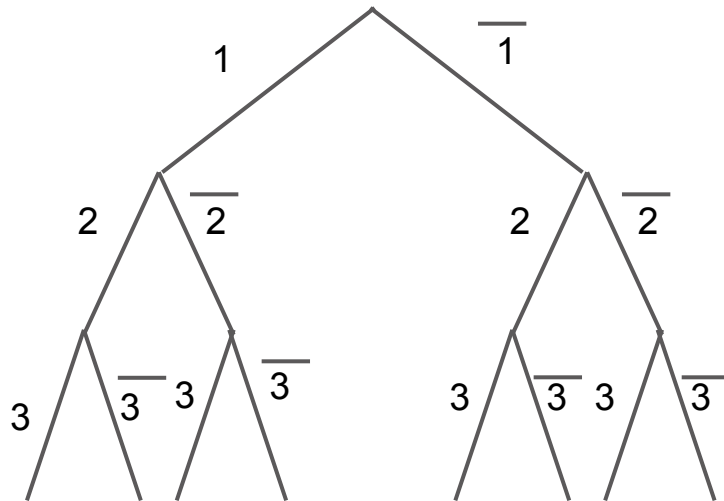
Néanmoins, la **recherche locale** en phase 2 [1994] permet d'obtenir une bonne approximation sur des instances de **tailles beaucoup plus importantes** (jusqu'à 400 sommets).

Inconvénient : cette variante est approchée. **Solution** : Prouver l'optimalité de la frontière (ou la compléter) avec un branch and bound multi-objectifs

Branch and bound multi-objectifs

Branch and bound multi-objectifs

- Fondé sur l'exploration d'un arbre d'énumération :



- Chaque noeud de l'arbre correspond à un sous-ensemble de solutions réalisables
-

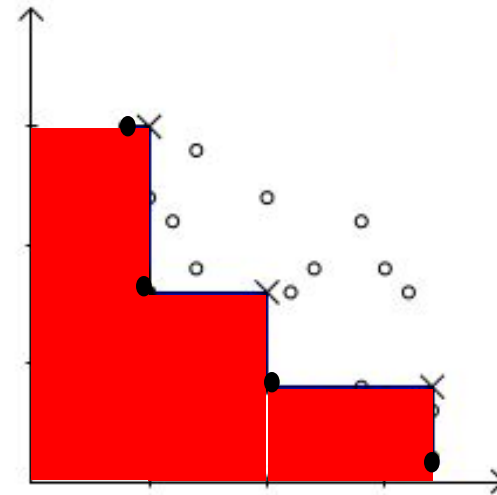
Branch and bound multi-objectifs

- L'idée principale est d'utiliser une **procédure de calcul de borne** pour explorer une portion la plus petite possible de l'arbre d'énumération :

si on peut prouver qu'aucune solution dans le sous-ensemble considéré ne pourra faire mieux que la meilleure solution trouvée jusqu'alors, alors il est inutile d'explorer le sous-arbre correspondant
 - Comment étendre ce principe au cas multi-objectifs ?
 - D'abord on doit calculer un **ensemble de Pareto approché** de bonne qualité, par exemple avec une recherche locale
-

Branch and bound multi-objectifs

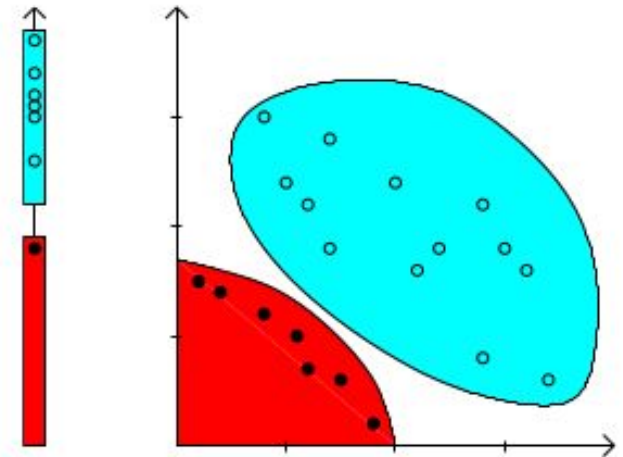
A l'issue de la
procédure, les points
non-dominés
appartiennent
nécessairement à la
zone rouge.



Branch and bound multi-objectifs

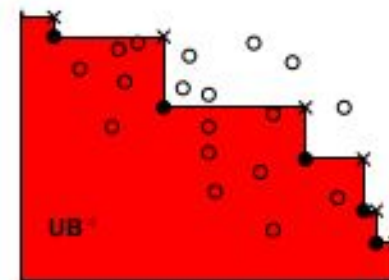
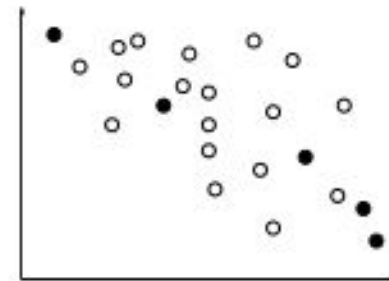
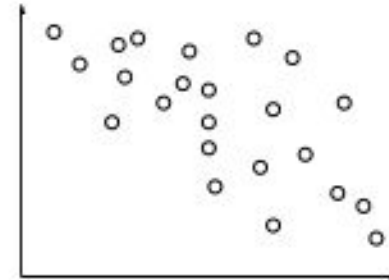
Comment tester si un sous-ensemble ne peut inclure de nouveaux points non-dominés ? → utiliser des **ensembles bornant**

- proposés par Villareal et Karwan (1981).
- Mis en oeuvre indépendamment par Ehrgott et Gandibleux (2007) et Sourd et Spanjaard (2008)



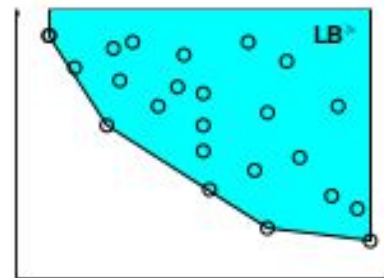
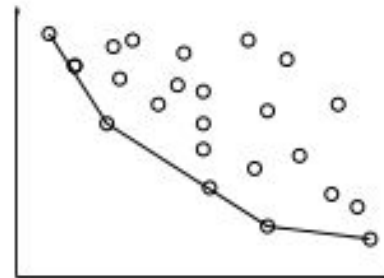
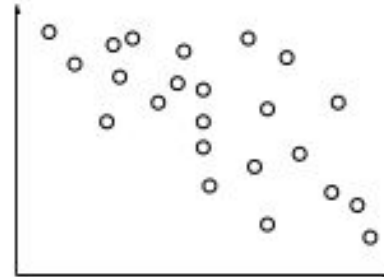
Branch and bound multi-objectifs

- Un **ensemble bornant supérieur** consiste en un ensemble d'images de solutions réalisables (les points noirs sur la figure)
- La **zone de recherche** est la portion de l'espace des objectifs qui n'est dominée par aucun des points de l'ensemble bornant supérieur (la **zone rouge** sur la figure)



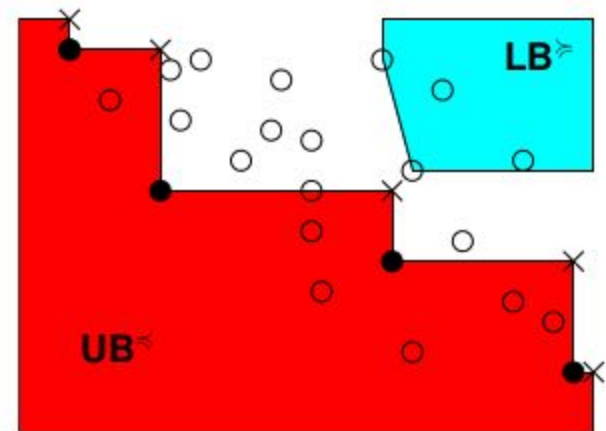
Branch and bound multi-objectifs

- Un **ensemble bornant inférieur** consiste en un ensemble LB de points tels que tout point réalisable est dominé au sens large par au moins un point de LB
- La **frontière non-dominée** de l'enveloppe convexe des points réalisables est un ensemble bornant inférieur valide
- Tous les points réalisables sont nécessairement au-dessus de l'ensemble bornant inférieur (la **zone bleue** sur la figure)



Branch and bound multi-objectifs

- En chaque noeud de l'arbre d'énumération, on teste la vacuité de l'intersection entre la **zone rouge** et la **zone bleue**
- Cela revient à tester si il n'y a pas de "croix" à l'intérieur de la **zone bleue**
- Dans ce cas, le sous-ensemble correspondant de solutions ne peut pas inclure de nouveau point non-dominé !



Branch and bound multi-objectifs

Une procédure de branch and bound multi-objectifs a été mise en oeuvre pour le problème de **l'arbre couvrant bi-objectifs** (Sourd and Spanjaard, 2008) et a permis de multiplier par 10 la taille des instances qui peuvent être résolue dans un temps raisonnable (de 40 à 400 sommets).

Une thèse (Lacour, 2014) présente une tentative d'étendre la méthode au cas à **tri-objectifs**, ce **qui rend l'approche plus délicate**.

Algorithmes approchés avec garantie de performance

La notion d' ε -couverture

Définition : ε -dominance

La relation d' ε -dominance entre vecteurs de $\mathbb{R}^q$ est définie par :

$$x \preceq_{\varepsilon} y \text{ si } [\forall i \in \{1, \dots, q\}, x_i \leq (1 + \varepsilon)y_i]$$

Définition : ε -couverture

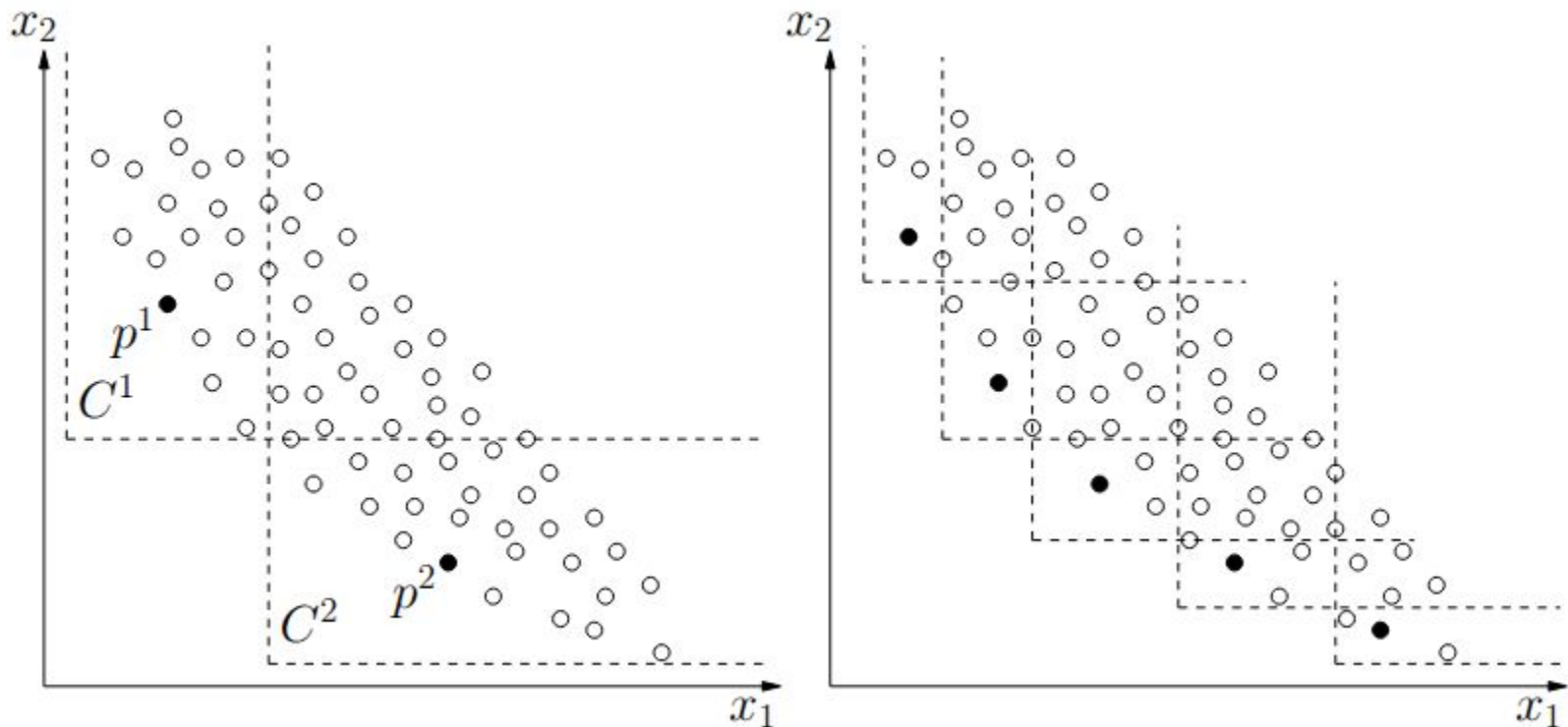
On dit qu'un ensemble Y **ε -couvre** X si pour tout vecteur $x \in X$ il existe $y \in Y$ tel que $y \preceq_{\varepsilon} x$. Si aucun sous-ensemble strict de Y n' ε -couvre X , on dit que Y est une **ε -couverture minimale**.

Cependant, plusieurs ε -couvertures minimales de tailles distinctes peuvent coexister.

Exemple : $x=(800,950)$, $y=(880,880)$, $z=(950,800)$, $\varepsilon=0.1$
 $\{x,z\}$ et $\{y\}$ sont des ε -couvertures minimales de $\{x,y,z\}$.

Frontière ε -approchée

Une **frontière ε -approchée** est une ε -couverture de la frontière de Pareto. Ci-dessous deux frontières ε -approchées pour deux valeurs distinctes de ε .



Passage à l'échelle logarithmique

Question Existe-t-il toujours une ε -couverture de cardinal réduit ?

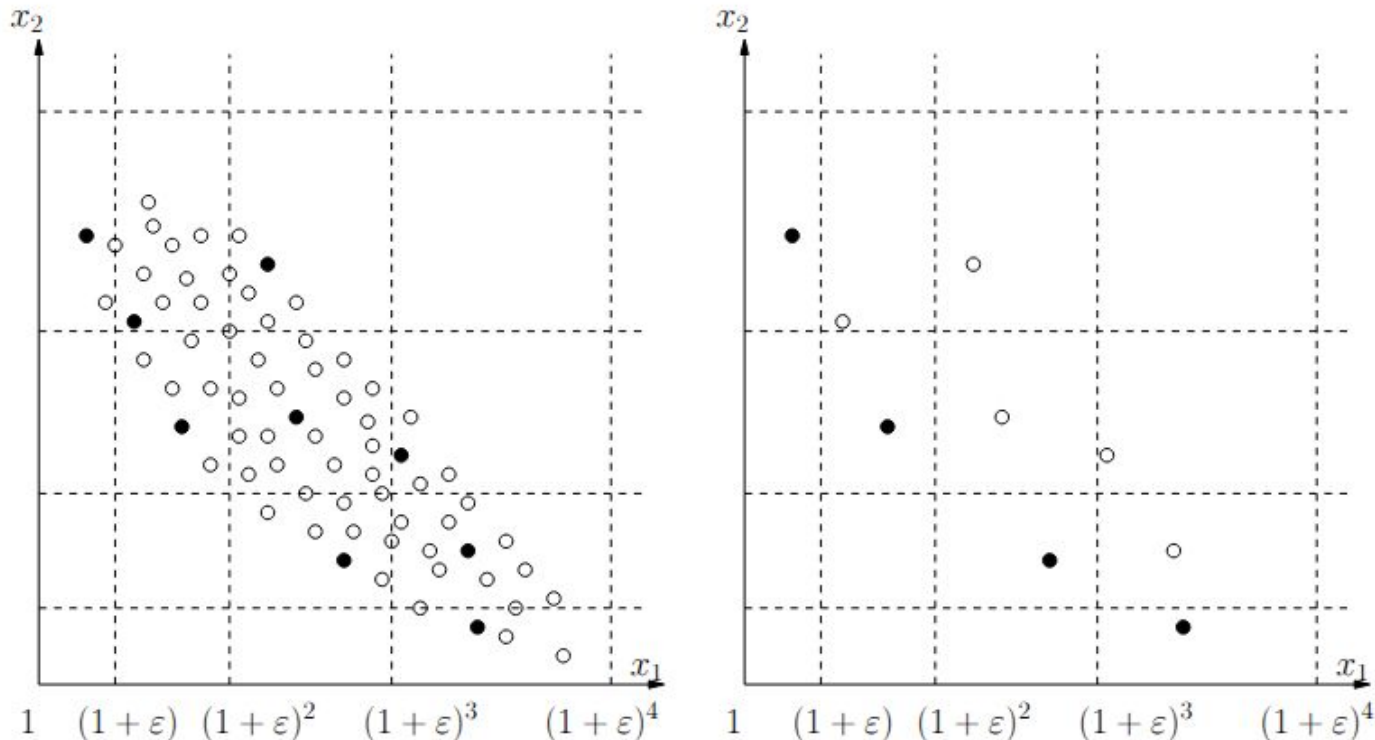
Théorème (Papadimitriou et Yannakakis, 2000)

Pour tout problème multi-objectifs, il existe une frontière ε -approchée de cardinal polynomial en $1/\varepsilon$ et en la taille de l'instance.

Idée de la preuve : passer sur une échelle logarithmique, c'est-à-dire un système de graduation en progression géométrique où chaque graduation est le produit par $(1+\varepsilon)$ de la graduation précédente.

La grille de l'échelle logarithmique

Les points dans une même case de la grille s' ε -dominent les uns les autres $\Rightarrow$ en prenant **un point par case non-dominée**, on obtient une frontière ε -approchée



La grille de l'échelle logarithmique

Si la valeur d'une solution sur un objectif est majorée par M , le nombre de gradations est majoré par k tel que :

$$(1 + \varepsilon)^k \leq M < (1 + \varepsilon)^{k+1}$$

$$k \leq \log_{(1+\varepsilon)} M < k + 1$$

$$k = \left\lfloor \frac{\log_2 M}{\log_2(1 + \varepsilon)} \right\rfloor$$

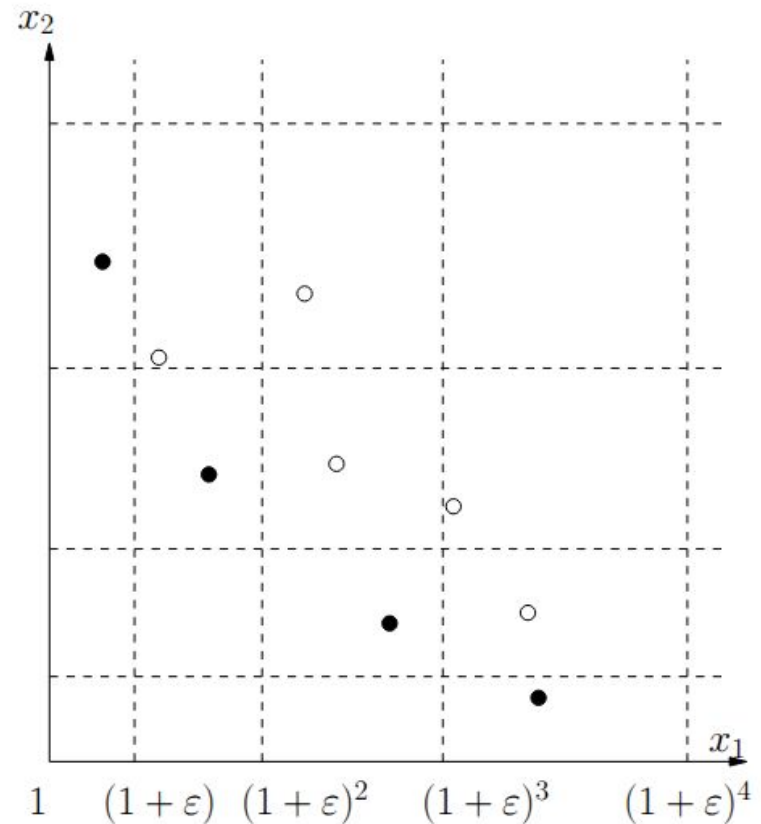
Pour q objectifs, le nombre de cases non-dominées de la grille est donc majoré par : $\left\lfloor \frac{\log_2 M}{\log_2(1 + \varepsilon)} \right\rfloor^{q-1}$

Pour $M \in O(2^n)$, où n est la taille de l'instance, ce nombre est en :

$$O\left(\left\lfloor \frac{n}{\log_2(1 + \varepsilon)} \right\rfloor^{q-1}\right)$$



Polynomial en n et $1/\varepsilon$ pour q fixé !

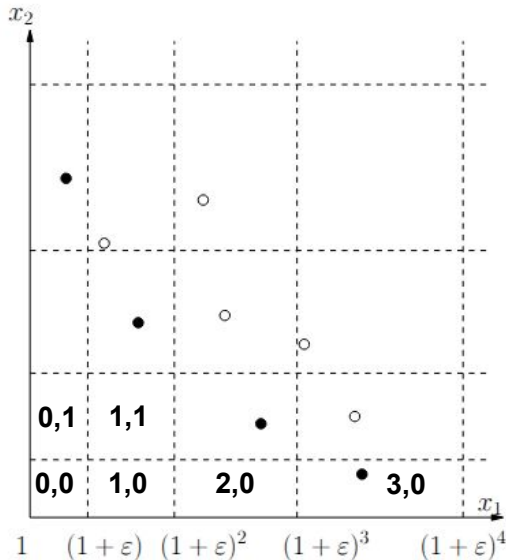


En pratique

$$x = (x_1, x_2)$$

$$\ell(x_1) = \left\lfloor \frac{\log x_1}{\log(1+\varepsilon)} \right\rfloor$$

$$\ell(x_2) = \left\lfloor \frac{\log x_2}{\log(1+\varepsilon)} \right\rfloor$$



$$\{(x, 2^{k+1} - x) : x \in \{1, \dots, 2^{k+1} - 1\}\}$$

Rappel Tous les éléments sont non-dominés par construction

Conséquence Quand k augmente, le nombre d'éléments non-dominés devient rapidement très grand

Mais prenons $k = 3$ et $\varepsilon = 1$

15 éléments non-dominés de $(1, 15)$ à $(15, 1)$

4 cases non-dominées distinctes de $(0, 3)$ à $(3, 0)$ sur la grille logarithmique !

Plus généralement le nombre de cases non-dominées est majoré par

$$\left\lfloor \frac{\log 2^{k+1}}{\log(1+\varepsilon)} \right\rfloor + 1$$

Exemple $k = 15$ et $\varepsilon = 0.1$

$$2^{16} = 65536 \text{ éléments non-dominés}$$

Nombre de cases non-dominées majoré par

$$\left\lfloor \frac{\log 65536}{\log(1.1)} \right\rfloor + 1 = 117$$

Deux points x et y sont **dans une même case** si $\ell(x) = \ell(y)$

De multiples algorithmes en temps polynomial ont été proposés pour calculer une approximation du front de Pareto avec n'importe quel ratio d'approximation (**schéma d'approximation**), notamment pour le **sac à dos multi-objectifs**.

Synthèse

Plusieurs procédures de résolution pour les problèmes combinatoires multi-objectifs, dont :

- programmation dynamique multi-objectifs,
- énumération ordonnée,
- méthode en deux phases,
- branch and bound multi-objectifs.

Il y en a d'autres ! (fondées sur la programmation mathématique par exemple)

Un des principaux verrous du domaine :

Les procédures de résolution présentées fonctionnent bien pour deux objectifs. Pour **trois objectifs ou plus**, les problèmes deviennent plus difficiles, et il reste beaucoup à faire.
